



MEMORIA PRÁCTICA FINAL

Rocío Muñoz del Olmo, Marco Ángel-Cruz
Carralero y Valeria Gonzalo Martínez

INTRODUCCIÓN

El juego creado se basa en el paso de un personaje, el jugador, por distintas habitaciones. En cada una de ellas se encuentra con diferentes objetos y enemigos. El usuario tiene que conseguir encontrar la salida de cada una de las salas para poder acceder a la siguiente; de esta forma, llegará a la última y, siguiendo este proceso, la salida de esta significará la victoria.

A la hora de programar el juego, hemos optado por la construcción de un Producto Mínimo Variable, priorizando la estabilidad y la arquitectura, así como la corrección algorítmica por encima de la sobrecarga de funcionalidades. A continuación, detallamos cómo ha sido el proceso de creación de juego, que se encuentra bajo el patrón Modelo-Vista-Controlador, persiste mediante JSON y está renderizado con JavaFX.

FASES DE PROGRAMACIÓN Y CREACIÓN DEL PROYECTO

Con el repositorio de GitHub creado, nos dividimos las tareas. De esta forma, tuvimos una organización previa a la hora de programar. La persona A se encargó de las estructuras, partiendo de las que ya habíamos creado en prácticas anteriores y mejorándolas y adaptándolas para esta. La persona B se centró en el paquete *model*. Creando las clases necesarias y siguiendo los diagramas UML. Por su parte, la persona C diseñó los bocetos de la interfaz para el mapa y la pantalla del jugador además de configurar el proyecto y conectar el esqueleto de JavaFX.

En la siguiente fase, la persona A desarrolló estructuras como *MyMatrix* para representar las habitaciones. También implementó un grafo y un Algoritmo de Búsqueda en Anchura, de esta forma, podía calcular el camino mínimo en las salas hasta encontrar la salida entre otras. La persona B, por otro lado, Implementó el movimiento ortogonal y el gestor de turnos. Además, integró estructuras para que el juego fuera iterativo. La persona C, hizo que el juego se cargase dinámicamente, utilizando JSON y conectó la matriz gráfica con el modelo.

Luego, la persona A terminó con el paquete de estructuras y construyó las pruebas unitarias con Junit, comprobando cada uno de los tests. La persona B se encargó de *CombatService* y de *InventoryService*. Así pues, estableció la lógica de recoger y equipar objetos entre otras. También implementó la IA de los enemigos para que se movieran por la habitación y atacasen al enemigo. La persona C, mediante el uso de JSON, consiguió que se pudieran guardar las partidas, dejando lista la interfaz de usuario.

Finalmente, para terminar el proyecto, la persona A redactó los comentarios y documentó el código. Mientras, la persona B realizó las pruebas necesarias jugando al propio juego y localizando los bugs, así como redactó la parte fundamental de la memoria. Por último, la persona C terminó con el diseño del software actualizando los diagramas UML, recopiló el diario de uso de la Inteligencia Artificial y empaquetó el proyecto para la entrega.

ESTRUCTURA DE CLASES, PROPÓSITO Y FUNCIONAMIENTO GENERAL

A la hora de estructurar el código fuente, dividimos el desarrollo en bloques modulares bien diferenciados. Entre ellos destacamos los siguientes:

1. Entidades del Juego (Capa del Modelo)

Estas clases funcionan como los contenedores lógicos que almacenan los atributos, variables de estado y recursos de los personajes esenciales de la partida:

Player<T>: Representa el modelo del héroe de la aventura. Al ser una clase genérica orientada a la interfaz Comparable, permite que el jugador interactúe y almacene cualquier tipo de ítem en su inventario. Su propósito es gestionar las estadísticas vitales (vida, ataque, defensa, dinero y movimiento), aplicando un control de límites estricto en sus métodos mutadores para impedir la corrupción de datos (como que la defensa caiga por debajo de cero o el movimiento sea inferior a uno).

Enemy: Modela las amenazas estándar del mapa. Su propósito es gestionar pools de salud duales (máxima y actual) para permitir dinámicas de regeneración. Su principal característica es la mitigación de impactos dentro de su método de recepción de daño, el cual introduce un factor de aleatoriedad mediante la generación de bits dinámicos (0 o 1) para simular esquivas o bloqueos críticos.

GameState: Funciona como el nexo central de datos o el "cerebro estático" de la sesión. Su propósito no es procesar acciones, sino registrar las variables globales en tiempo real: almacena las instancias del jugador y el enemigo, sus coordenadas de filas y columnas exactas, el contador de turnos acumulados y la referencia directa al grafo que compone el mapa completo.

2. Infraestructura Espacial y Red de Datos

Room<T>: Modela el entorno físico por el que camina el jugador. Envuelve nuestra clase personalizada MyMatrix para estructurar una cuadrícula bidimensional. Al instanciarse, pobla automáticamente cada coordenada con objetos Cell, los cuales actúan como contenedores genéricos capaces de albergar personajes, puertas (Door) u objetos interactivables, controlando de forma segura la ocupación espacial.

AristaOBJ y VerticeOBJ<T>: Constituyen los elementos atómicos de la red de transporte. El vértice funciona como el nodo del mapa que resguarda el dato genérico de la habitación (Room), mientras que la arista actúa como el puente direccional entre salas, implementando una ordenación alfabética mediante su predicado de texto para facilitar búsquedas avanzadas en la red.

GrafoOBJ<T>: Coordina la topología completa del videojuego. Su propósito es gestionar la colección global de vértices y consolidar las conexiones válidas entre ellos, verificando estrictamente mediante contratos de tipado genérico que los datos de origen y destino coincidan antes de establecer un camino transitable.

3. Capa de Servicios Lógicos (Controladores de Acción)

Estas clases carecen de atributos persistentes y operan de forma pura procesando la física, el entorno y las reglas del juego mediante algoritmos secuenciales:

MovementService: Controla el desplazamiento. Analiza la matriz circundante del GameState para devolver listas enlazadas (ListaSE) con las direcciones seguras (celdas no ocupadas). Gestiona el algoritmo de movimiento libre de la IA enemiga y procesa el evento crítico de colisión con puertas, operando la transición switchRoom al extraer la habitación de destino directamente desde el grafo.

CombatService: Resuelve las hostilidades del juego. Escanea los cuatro ejes ortogonales (arriba, abajo, izquierda, derecha) buscando instancias explícitas de entidades rivales. Su propósito es habilitar los comandos de ataque, canalizar el cálculo de daño recíproco entre las entidades adyacentes y limpiar las celdas de la matriz cuando la salud de un enemigo cae a cero.

InventoryService<T>: Rige la economía de recolección de la partida. Su funcionamiento consiste en discriminar positivamente cualquier elemento ocupado en la matriz que no responda a la identidad de un enemigo. Captura los datos genéricos del suelo, los transfiere de forma segura al inventario del jugador como estructuras comparables y libera el espacio de la celda para simular la recogida física del ítem.

De esta forma, la arquitectura del software asegura un flujo circular perfecto: los servicios leen las coordenadas espaciales de la habitación, comprueban las reglas de negocio sobre los datos del modelo y actualizan el estado global en el grafo, manteniendo el sistema robusto, escalable y completamente libre de acoplamientos innecesarios.

CONCLUSIÓN

De esta forma, podemos llegar a la conclusión de que la planificación inicial lo es todo. Así como también ha sido fundamental no poder usar java.util. También ha sido muy importante basarnos en un MVP e ir actualizando y compartiendo el código en GitHub.

CRÍTICA DEL PROYECTO

A pesar del reparto de tareas acorde a las capacidades de cada miembro del grupo, el cumplimiento de plazos se ha dificultado. Luego podríamos mejorar la gestión del tiempo. Sin embargo, destacamos la buena organización inicial que nos ha permitido trabajar de forma independiente. De la mano con esto, la comunicación en el equipo es clave y, si bien al principio nos ha costado algo más, a medida que hemos avanzado en el proyecto, íbamos actualizando el estado de nuestras tareas y poniendo en común el proceso.

Por otro lado, se podría mejorar el entendimiento de JSON y JavaFX, así, en futuros trabajos conseguiremos desenvolvernos mejor. Sin embargo, no hemos sido ambiciosos a la hora de programar y hemos creado un programa sencillo. De esta forma, hemos conseguido cumplir con los requisitos necesarios sin complicar el código. Además, en cuanto a la estructura de este, lo hemos organizado por paquetes creando una jerarquía que ha ayudado a la organización y visualización del programa.

